# Final Report - DSCI 351 Project

## Introduction:

In this project, we decided to create an inventory management system for a trader who owns an inventory. The trader can buy products from a supplier and sell products to a retailer. The app allows the trader to input the details of the product(s) it has purchased from a supplier. After the product(s) has been purchased, it is added to the trader's inventory. Then, the app allows the trader to receive an order for a product from a retailer. The trader can input the retailer's order into the system, and once inputted, the app subtracts the product ordered from the inventory so that it can supply this product to the retailer. Overall, this system will allow a trader to manage their entire inventory's buys and sales through one application making it easy for them to track their financials and accounts.

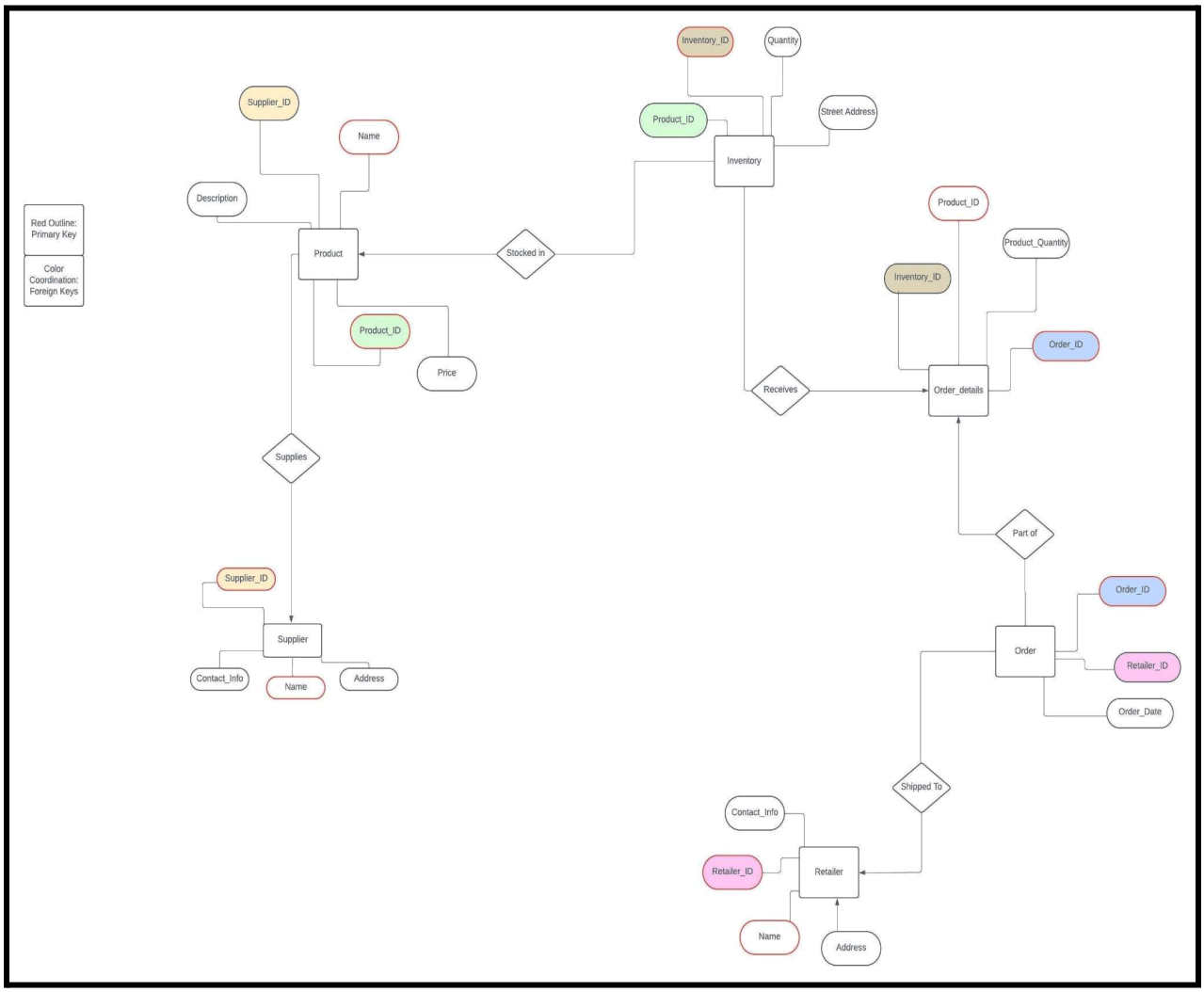
## Planned Implementation:

We have come a long way from the project proposal to the final project we created. In the project proposal, we considered several ideas about what function our app should perform. Initially, we had set out the goal of product management functions, inventory level tracking, transaction logging, and reporting on inventory data. We believe that the final app we ended up with performed these functions and more. For instance, the app allows for CRUD functionality – as well as an added "Display all" functionality which displays a full table of the selected entity that we thought can be useful – we can also track the level of inventory in the app. However, we did not put a cap on the amount of inventory that could fit in a particular warehouse to allow for scalability. We recorded both the buy and the sell side for a trading business, and provided the user the ability to perform NoSQL queries directly in either MongoDB and RedisDB.

After the project proposal phase, we began creating a step-by-step plan on how to create the app. We broke down the task into several pieces and worked on them individually. Since the inventory management system contains data on both the buy side of a trader's business, and the sell side, we decided to use MongoDB to store data on the Sell-Side and Redis to store data on the Buy-Side. This helped us compartmentalize our app. We started working with MongoDB as we were most familiar with it. Here, we further broke down the task by creating an ER diagram to represent the entities that store data, the attributes they contain, and their relations with one another. We did the same on the buy side for Redis.

Overall, the approach we used to create the app was a top down approach. Here, we visualized what the final app would look like and then we broke it down into smaller pieces so that we could successfully create it. Finally, we went beyond the goal set out by the project proposal and decided to create a GUI for the app. This was the icing on the cake and allowed the application to be practically used.

# Architecture Design:




This ER Diagram represents the architecture of the app. Here, the buy side of the inventory management system includes the supplier (who is the maker of a product) and the product entities. The data for these two entities is stored in RedisDB. The Sell side data which includes the inventory, order_details, order, and retailer entities is stored in MongoDB.

The types of relations between the entities is described below

1) Supplier supplies product(s): this is a one to many relationship. A single supplier can supply multiple products but a single product is supplied by a single supplier.
2) Product stocked in Inventory(s): this is a one to many relationship. A single product type can be stored in multiple inventory locations but a single inventory index contains a single product.

3) Inventory(s) receive order_detail: this is a many to one relationship. Multiple inventory records can be contain a single order_detail but a single order detail (presuming that each order detail refers to an item line in an order which can be fulfilled from multiple inventory locations)
4) Order_detail is part of Order: this is a many to one relationship. Many order details can be a part of a single order but a single order can contain multiple order details.
5) Order shipped to Retailer: The relationship "Shipped To" is a many-to-one relationship. Many orders can be shipped to a single retailer, but each order is shipped to only one retailer.

This flow helps the trader in charge of using the inventory management system understand how to use the app. The flow goes as such:
First, the trader purchases a product(s) from the supplier.

1) The supplier then supplies the product to the trader.
2) The product is then added to the trader's inventory.

Next, the retailer places an order with the trader to buy a product

1) Here, the retailer places an order of one or multiple products
2) The order contains an order detail for the individual products they purchase
3) The order detail is then received by the trader. Then, if the product already exists in the inventory, the quantity of the product ordered by the retailer is subtracted from the quantity of product available in the inventory. If the product does not exist, proper and clear error/exception handling is used.

# Implementation

**Functionalities:**
The "FinalInventoryManagementApp" is designed to streamline inventory management processes for businesses, offering comprehensive features that allow users to manage various aspects of their operations efficiently:

- CRUD Operations: Users can create, read, update, and delete (CRUD) information related to inventory, orders, retailers, products, and suppliers.
- Data Visualization: The application provides an intuitive graphical user interface (GUI) that displays data in tabular forms, making it easier for users to view and manage large datasets.
- Real-time Data Access: Integration with Redis allows for fast access to frequently used data such as product and supplier details, enhancing performance and user experience.
- Automated Data Synchronization: Changes made in the system through the GUI are automatically synchronized with the MongoDB and Redis databases, ensuring data consistency across the platform.

Furthermore, the backend of the app (created using python) has the following structure:

- **db.py**
  The `db.py` file contains a suite of functions designed to interact with the database for an Inventory Management system. These functions are integral to the system's operations,

whether accessed through the Inventory Management GUI or used programmatically in automated scripts for batch processing and maintenance tasks. The functionalities provided by these functions include updating existing records, such as modifying the quantity of an item in the inventory (`update_inventory_quantity`) or altering the details of an order (`update_order_detail`). Additionally, the file includes capabilities for deleting records, with specific functions to remove inventory (`delete_inventory`), orders (`delete_order`), retailers (`delete_retailer`), and order details (`delete_order_detail`). Beyond updating and deletion, the `db.py` also facilitates data retrieval, offering functions to fetch complete lists of inventory (`get_all_inventory`), orders (`get_all_orders`), retailers (`get_all_retailers`), and order details (`get_all_order_details`). These comprehensive data manipulation and retrieval tools are essential for effective management and operation of the inventory system.

- **gui.py**
  The `gui.py` file serves as the foundational module for the graphical user interface of an Inventory Management System, utilizing the `tkinter` and `customtkinter` libraries to craft a user-friendly platform. This GUI is designed to facilitate CRUD (Create, Read, Update, Delete) operations across various data entities such as inventory, orders, products, suppliers, and retailers, making it an essential tool for efficient inventory management.

  Within this module, the primary class is the `InventoryManagementGUI`, which orchestrates the setup and functionality of the GUI. This class includes several key functions:
  - `__init__(self, root)`: Establishes the main window and its components, laying the groundwork for the interface.
  - `on_action_selected(self, event=None)`: Manages actions chosen from dropdown menus, ensuring the system responds to user selections.
  - `get_display_function(self, entity)`: Determines and retrieves the display function corresponding to the selected database entity.
  - `confirm_selection(self)`: Validates the choices made by the user and sets up the input fields accordingly.
  - `generate_input_fields(self)`: Creates input fields dynamically, tailored to the specific action and entity involved.
  - `submit(self)`: Executes the input data handling and carries out the action chosen for the selected entity.
  - `create_display_window(self, title, columns)`: Opens a new window to display entity records in a structured tabular format.
  - Entity-specific display functions like `show_all_inventory()`, `show_all_orders()`, etc., each tailored to present comprehensive listings of their respective records.

  To use this GUI, one simply runs the `gui.py` script, which launches the interface, assuming that the necessary backend services (MongoDB and Redis) are active and correctly configured to link with the `db.py` and `redis_db.py` modules. This script integrates deeply with its backend, relying on the Python libraries `tkinter` and

`customtkinter` for the GUI elements, and it requires that these dependencies are updated, especially in contexts using Python versions earlier than 3.7, to ensure compatibility with `customtkinter`.

- **redis_db.py**
  The `redis_db.py` module is an integral component of an Inventory Management System, specifically focused on interfacing with Redis to manage product and supplier data. This module is dedicated to encapsulating comprehensive CRUD (Create, Read, Update, Delete) operations, facilitating direct manipulation and retrieval of records stored within Redis.

  Key functionalities provided by this module include:
  - **Creation Functions**: `create_product` and `create_supplier` are used to add new product and supplier records into Redis, respectively. These functions ensure that new entities are seamlessly integrated into the existing database structure.
  - **Retrieval Functions**: `retrieve_product` and `retrieve_supplier` fetch specific records based on defined parameters, allowing for targeted data access.
  - **Update Functions**: `update_product` and `update_supplier` facilitate modifications to existing records, ensuring the data remains current and accurate.
  - **Deletion Functions**: `delete_product` and `delete_supplier` remove records from Redis, enabling clean data management and ensuring that outdated or unnecessary information is expunged.
  - **Bulk Retrieval Functions**: `get_all_products` and `get_all_suppliers` provide comprehensive access to all records of products and suppliers stored in Redis, which is crucial for operations requiring a broad overview of inventory or supplier landscapes.

  This module is designed to be utilized within an Inventory Management GUI or other system components that necessitate interaction with Redis for effective management of product and supplier information. The functionality of this module relies on `redis-py`, the Redis Python client, which provides straightforward and efficient access to Redis operations through Python. This dependency is critical as it bridges the gap between Redis database operations and the Inventory Management System's needs, ensuring that all interactions with Redis are executed smoothly and efficiently.

- **redisQueries.py**
  The `redisQueries.py` module is a specialized component of an Inventory Management System designed to perform specific queries on a Redis database. It includes functions such as `find_supplier_with_most_expensive_product`, which identifies the supplier offering the most expensive product, and `list_products_by_supplier`, which generates a list of all products supplied by a specified supplier. These functions are crucial for strategic business analyses, such as assessing product value distribution among suppliers and monitoring supply chain efficiency. To utilize these functions, users need to ensure that their working directory is set to where `redisQueries.py` resides, or have the directory added to their Python path for easy access. Additionally, it is imperative to

verify that the Redis server is active and properly configured to connect with the necessary host and port as defined in the script. This setup guarantees that the system can retrieve and manipulate data effectively, providing valuable insights through targeted queries within the Redis environment.

- **models.py**
  The `models.py` module within the FinalInventoryManagementApp is fundamental for defining and managing the data models used across the application. Leveraging mongoengine, a Document-Object Mapping (DOM) library specifically designed for MongoDB, this module establishes structured schemas for various data entities crucial to the app's functionality. These include:

  - **Retailer**: Captures details about a retailer, such as ID, name, location, and contact information.
  - **Order**: Describes an order placed by a retailer, detailing the order ID, the associated retailer, and the date the order was made.
  - **Inventory**: Defines an inventory item by ID, linking it to a product ID, and noting its quantity and storage location.
  - **OrderDetails**: Maps the specifics of an order to the products included, quantifying each item within the order.
  - **Product**: Characterizes a product with attributes like ID, name, description, and price.

  These models are vital for the operational integrity of the FinalInventoryManagementApp, facilitating efficient data management and retrieval across both the application's GUI and its backend processes. They enable robust interactions within the database, allowing for the creation, manipulation, and retrieval of structured data critical for the application's daily functions. For instance, the process of adding a new retailer to the database involves creating a Retailer instance with specific attributes and saving this new entry, thus demonstrating the practical application of these models in real-world operations.

**Tech Stack:**

1) MongoDB: Used as the primary database to store detailed records of inventory, orders, and other entities. It is chosen for its flexibility with document schemas and scalability.
2) Redis: Employed for caching and quick retrieval of frequently accessed data, such as product details and supplier information, to enhance performance.
3) Python: The primary programming language used to build the application logic, chosen for its simplicity and the powerful libraries it supports.
4) mongoengine: A Python Object-Document Mapper (ODM) for MongoDB, used for defining data models and handling database operations.
5) Tkinter and customtkinter: These libraries are used to develop the application's GUI, providing a simple yet powerful interface for interacting with the system.

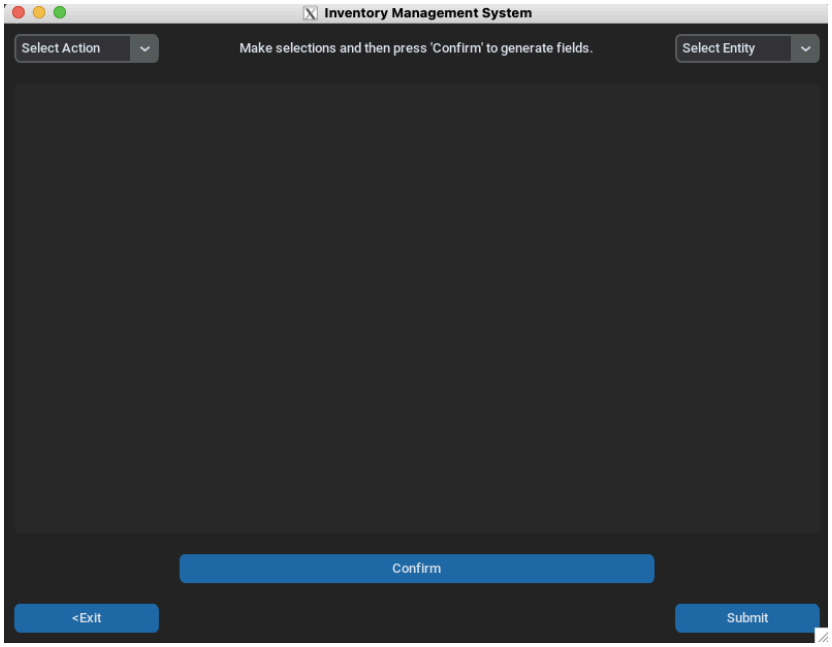
6) Redis-py: This library enables Python applications to work with Redis, used for managing Redis database operations within the app.

**Implementation Process:**

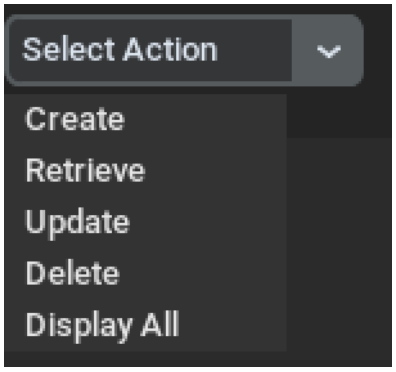
1. Designing Data Models: Initially, data models for retailers, orders, inventory items, order details, and products were designed using mongoengine. These models define the structure of documents in MongoDB.
2. Setting up Databases: MongoDB was set up to store the structured data, while Redis was configured to cache frequently accessed data like product and supplier details.
3. Developing the Backend: Python functions were implemented to handle CRUD operations for the various entities. These functions interact with MongoDB and Redis through mongoengine and redis-py, respectively.
4. Creating the GUI: The GUI was developed using Tkinter and customtkinter, which provided a user-friendly environment for performing CRUD operations. Dropdown menus, buttons, and input fields were set up to capture user inputs and display data.
5. Integration and Testing: After setting up the backend and GUI, extensive integration tests were conducted to ensure that all parts of the application worked together seamlessly. This included testing the synchronization between the MongoDB and Redis databases.
6. Deployment and Monitoring: The final step involved deploying the application on a suitable platform and setting up monitoring tools to ensure its performance and stability.

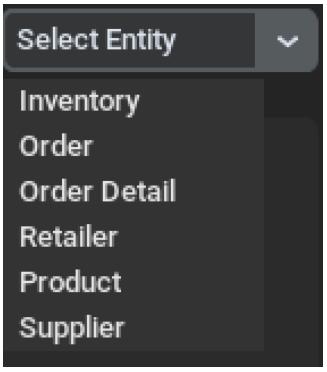
**Implementation Screenshots:**

1) **Homescreen:** this is the screen that appears when you open the application. Type in python3 gui.py to enter the application.

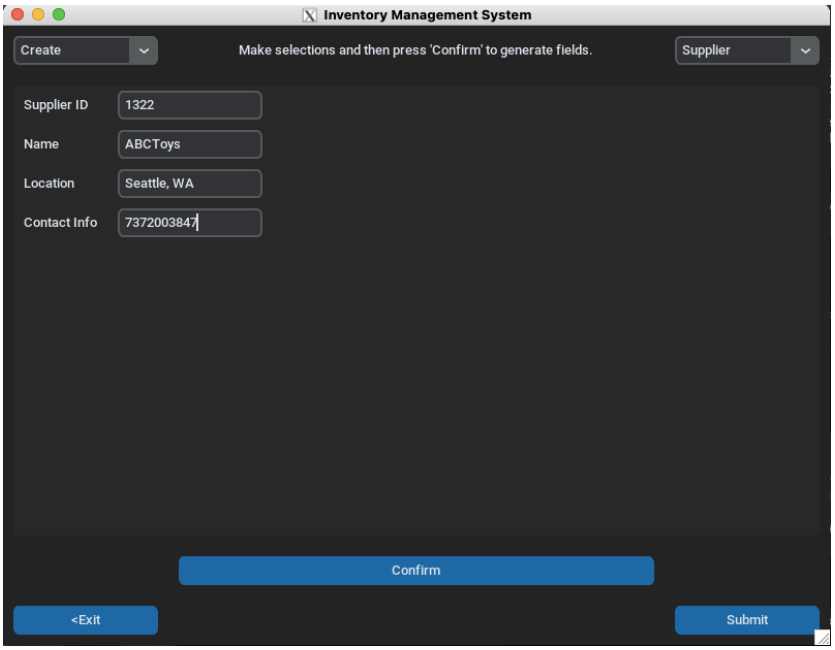


2) **Dropdown menu options:** this includes the CRUD functions in the ‘Select Action’ dropdown on the left and ‘Select Entity’ includes the Entities to perform the CRUD ›functions on the right.



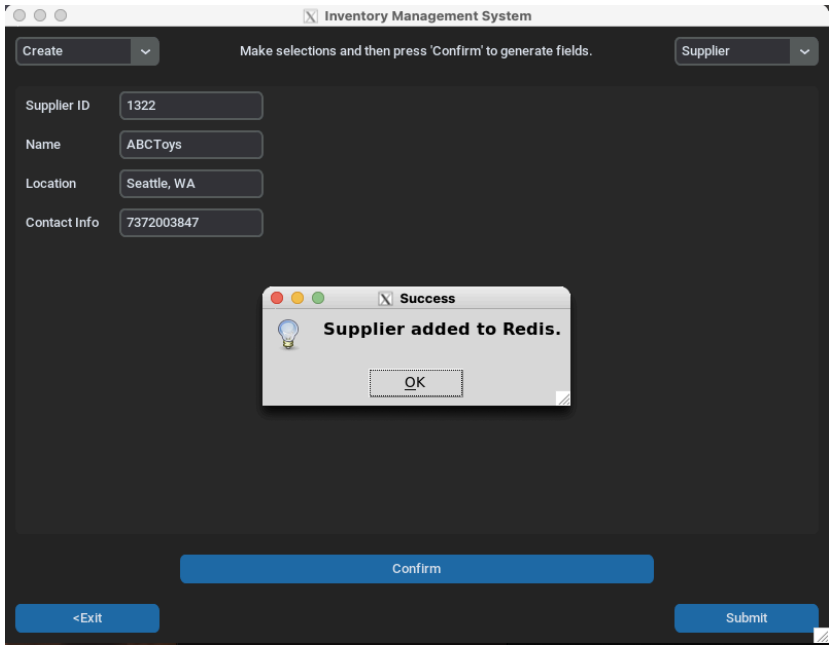




3) **Performing an Action and selecting an Entity:** the action being performed here is ‘Create’ and the Entity that is being created is ‘Supplier’. As can be seen from the screenshot below, the supplier information is entered into the input boxes. Next, we will

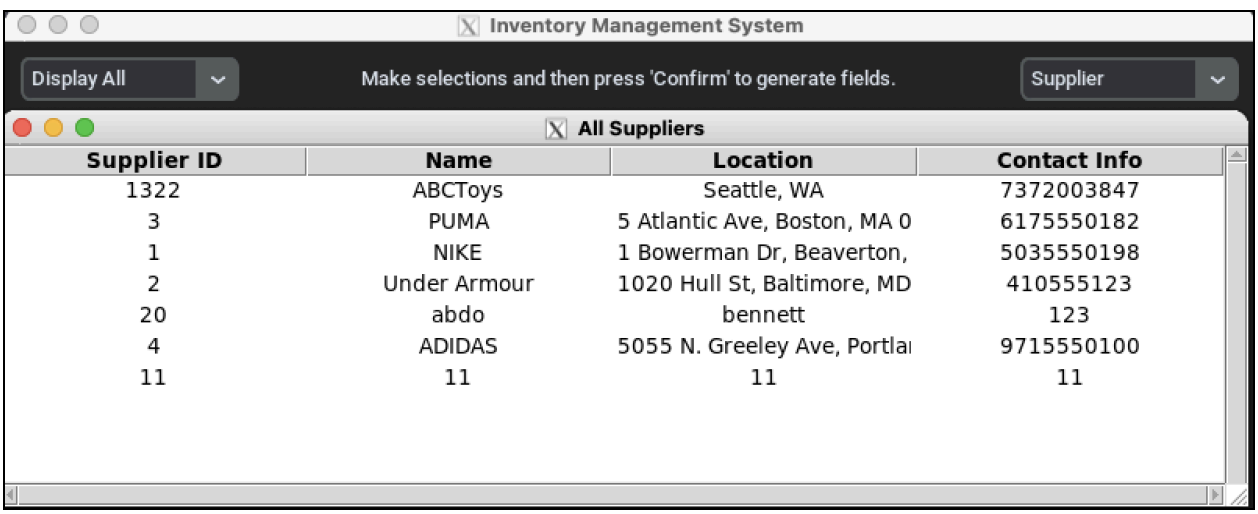



press submit to save the information.

4) **Successfully saved data:** after clicking the ‘Submit’ button, the supplier information was stored successfully in the redisDB database.




5) **Displaying the Successful Storage of data:** Here, after confirming the 'Display All' action, we can see the data has been successfully saved into the Redis Database.




| Supplier ID | Name | Location | Contact Info |
|---|---|---|---|
| 1322 | ABCToys | Seattle, WA | 7372003847 |
| 3 | PUMA | 5 Atlantic Ave, Boston, MA 0 | 6175550182 |
| 1 | NIKE | 1 Bowerman Dr, Beaverton, | 5035550198 |
| 2 | Under Armour | 1020 Hull St, Baltimore, MD | 410555123 |
| 20 | abdo | bennett | 123 |
| 4 | ADIDAS | 5055 N. Greeley Ave, Portla | 9715550100 |
| 11 | 11 | 11 | 11 |

**Queries:**

After Creating the architecture and backend for the project, we tested some queries using MongoDB and RedisDB

| To Find: | Database: | To Find: |
|---|---|---|

| | | |
|---|---|---|
| Find productIDs where retailerID is 2 | Mongo DB | db.order.aggregate([{$match: {retailer: '2'}}, {$lookup: {from: "order_details", localField: "_id", foreignField: "order", as: "order_details"}}, {$unwind: "$order_details"}, {$project: {_id: 0, product_id: "$order_details.product_id"}}]).toArray() |
| Find the name(s) and quantity of product ordered by the retailer(s) who have ordered productid = 2 | Mongo DB | db.order_details.aggregate([{$match: {product_id: '2'}}, {$lookup: {from: "order", localField: "order", foreignField: "_id", as: "order_info"}}, {$unwind: "$order_info"}, {$lookup: {from: "retailer", localField: "order_info.retailer", foreignField: "_id", as: "retailer_info"}}, {$unwind: "$retailer_info"}, {$project: {_id: 0, retailer_name: "$retailer_info.name", quantity: "$quantity"}}]).toArray() |
| Find inventory location for the products ordered by retailer name = 'Venice Beach Athletics' | Mongo DB | db.retailer.aggregate([{$match: {name: 'Venice Beach Athletics'}}, {$lookup: {from: "order", localField: "_id", foreignField: "retailer", as: "orders"}}, {$unwind: "$orders"}, {$lookup: {from: "order_details", localField: "orders._id", foreignField: "order", as: "order_details"}}, {$unwind: "$order_details"}, {$lookup: {from: "inventory", localField: "order_details.product_id", foreignField: "product_id", as: "product_inventory"}}, {$unwind: "$product_inventory"}, {$group: {_id: "$order_details.product_id", inventory_location: {$first: "$product_inventory.location"}}}, {$project: {_id: 0, product_id: "$_id", inventory_location: 1}}]).toArray() |
| 1. List products by supplier<br>2. Find supplier with the most expensive product<br>3. Exit | Redis | Run: python3 redisQueries.py |

# Learning Outcomes (Challenges Faced):

1) When we first started working on the project, we began with the intention of using Apache Cassandra and MongoDB as the NoSQL databases. While we did stick with MongoDB, we switched from using Apache Cassandra to RedisDB. This was primarily due to the lack of space available on our EC2 instance. Thus, even though we did download and try to run apache cassandra, we kept receiving a storage error so we couldn't input any data into the database. Since we were planning the database to retrieve data at a first pace and have enough flexibility to scale up the storage space of data, we chose Redis. Redis stores its data in the main RAM, which provides extremely fast access times, making it ideal for scenarios where high performance is critical.

Additionally, at its core, Redis is a key-value store, making it easy to store and manage data (one of the main functions of the application).

2) The second issue we encountered was working with the EC2 instance on the terminal. While writing large amounts of python code using the Nano function, we found it difficult to debug and edit the code. It made troubleshooting difficult. Thus, we wrote all our python code on Pycharm where we could easily edit and debug errors in the code, and then we pasted the code onto the python in the EC2 instance to test it out.
3) The last challenge we faced was creating the GUI. Since neither of us have worked with a GUI before, we decided to use tkinter (a basic gui). We used the customtkinter library to make the application more user friendly and aesthetically pleasing. However, at first, we only tried to implement the gui after coding the entire application. This made it very difficult for us to integrate the gui in the existing python files we created for the app's functionalities. Thus, we had to rewrite and reshape our existing python files. We initially had an app.py python pile that executed the functions of the app. We deleted this file and created a gui.py file to allow the gui to take user inputs and then execute the code to carry out the functions. This fix took a long period of time and a ton of research as neither of us have extensive python knowledge and only one of us has intermediate C/C++ knowledge.

# Individual Contributions:

**Dev**: I looked into the top down plan of creating the app including choosing the databases required to store the data in the inventory management system and creating the application's architecture. Additionally, I created NoSQL queries in both MongoDB and RedisDB to retrieve essential information and analytics required for the firm's function.

**Mazen:** As the primary developer, I established the foundational EC2 instance and actively participated in strategic discussions with Dev to ensure our project goals were effectively met. I was responsible for converting the initial plans and ER diagram into operational Python code, bridging the gap between theoretical designs and practical implementation.

Dev researched proper GUIs for our purposes and I took on learning and integrating the customtkinter library, chosen for its user-friendly and adaptable features, which enhanced the application's interface. My role involved programming the backend logic and ensuring the frontend GUI was seamlessly connected, facilitating a smooth user experience.

Together with Dev, I engaged in rigorous testing of the application. This collaborative effort helped us pinpoint and rectify various issues, improving the overall functionality and reliability of our project. My focus was on fine-tuning the code to resolve these issues, ensuring that each component functioned flawlessly within the broader system.

# Conclusion:

To conclude, this app is a basic inventory management application that can be used by a trader to track their inventory levels. It comprehensively uses 2 databases which allow for both the quick retrieval and access to data from RedisDB, and the ability to scale the inventory storage through MongoDB. Additionally, it is created in a user-friendly manner that allows the trader to

make changes to the information they have imputed into the databases with the help of the CRUD functions (create entity, retrieve entity, update entity, and delete entity).

# Future Scope:

Overall, we believe that this application has a large room for improvement. If we had more time, I believe that smaller details of the app. These include:

1) Compartmentalization of the storage of products within the inventory based on the product types can be implemented. For instance, to allow for easier access to products, we want to be able to search (query) for products by product types and find information about the retailers and suppliers through this method.
2) Additionally, we can also implement a function to monitor the trader's daily profit and loss through the form of a P&L. Then information can be logged into a new entity to track the trader's revenue. For instance, this will include revenue through assets like sales and value of warehouse land. It will also include costs such as cost of purchases, cost of capital, cost of owning the warehouse etc. this has the scope of housing and updating the trader's financial statements. This will also give the trader information about the historical product prices and the retailers and suppliers that offer the best prices for a particular product. Most importantly, it can explain the profit margins for a product.

**Dev Kapashi and Mazen Aboulela**